

Lista 6 - Shell Avançado II - Bash como Linguagem de Programação

Exercício 1

Crie o arquivo `ficha_robo.sh`. O script deve armazenar em variáveis o seu nome, curso e cidade, e receber como argumentos o nome de um robô e o seu nível de bateria. Com esses dados, exibe primeiro uma ficha do operador e em seguida o status do robô com a cor correspondente ao nível de bateria.

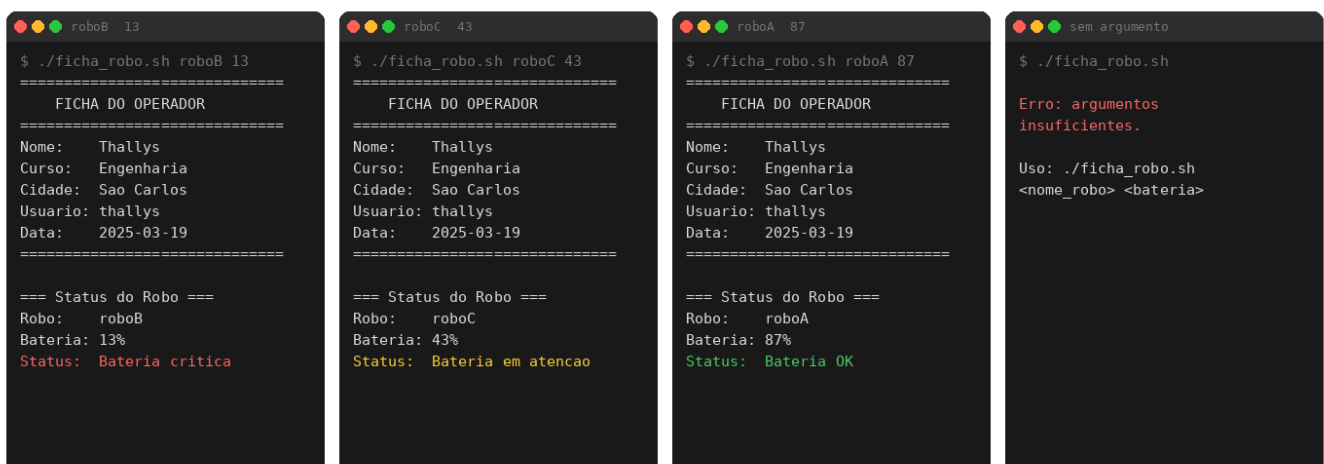
O script deve validar se os dois argumentos foram fornecidos. Se não, exibe uma mensagem de erro e encerra:

```
Erro: argumentos insuficientes.  
Uso: ./ficha_robo.sh <nome_robo> <bateria>
```

A classificação de bateria segue a tabela abaixo:

Bateria	Cor	Mensagem
80% ou mais	Verde	Bateria OK
Entre 20% e 79%	Amarelo	Bateria em atenção
Abaixo de 20%	Vermelho	Bateria crítica

A imagem abaixo mostra os quatro cenários possíveis de execução: bateria crítica, bateria em atenção, bateria OK e o erro de validação quando nenhum argumento é passado.



Dica: defina as cores como variáveis no início do script para deixar o `echo` mais legível.

Exercício 2

Crie o arquivo `frota_completa.sh`. O script deve armazenar em três arrays os nomes de quatro robôs, seus respectivos níveis de bateria e a tarefa atual de cada um. O script deve percorrer os três arrays ao mesmo tempo e exibir uma tabela formatada com todas as informações, incluindo o status colorido conforme o nível de bateria.

Ao final da tabela, exiba um resumo contando quantos robôs estão em cada categoria:

Robo	Tarefa	Bateria	Status
roboA	patrulha	87%	OK
roboB	recarga	15%	Crítico
roboC	mapeamento	43%	Atenção
roboZ	coleta	95%	OK

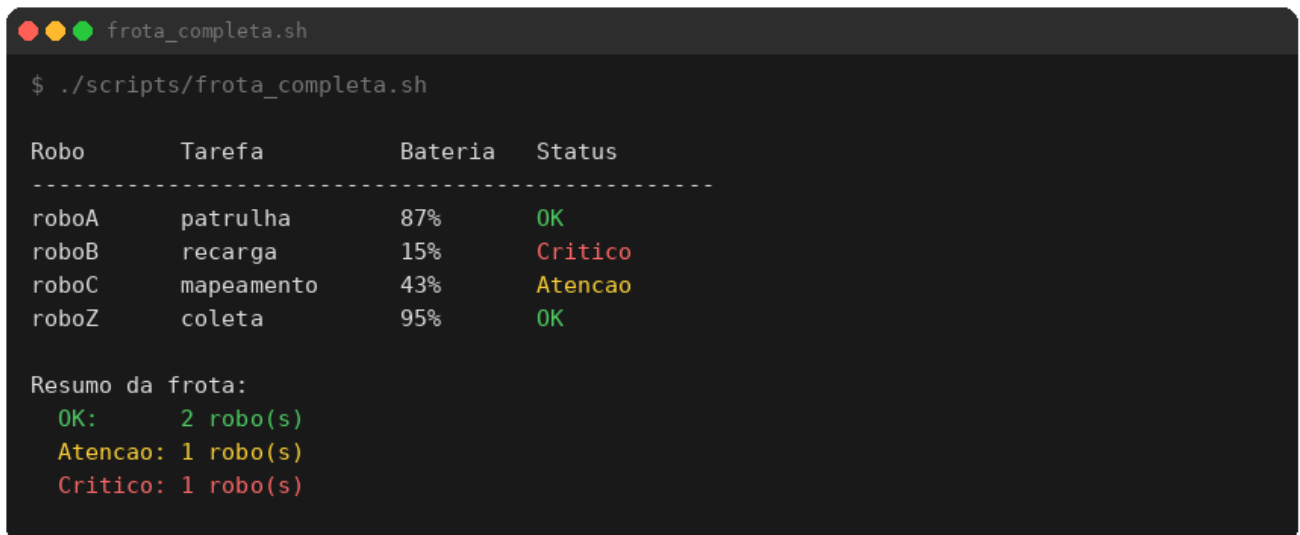
Resumo da frota:

OK: 2 robo(s)

Atenção: 1 robo(s)

Crítico: 1 robo(s)

Use a mesma lógica de classificação do Exercício 1: acima de 80% é OK, entre 20% e 79% é Atenção, abaixo de 20% é Crítico. Cada status e cada linha do resumo devem aparecer com a cor correspondente no terminal, conforme a imagem abaixo:



```
frota_completa.sh
$ ./scripts/frota_completa.sh

Robo      Tarefa      Bateria      Status
-----
roboA     patrulha     87%          OK
roboB     recarga      15%          Critico
roboC     mapeamento  43%          Atencao
roboZ     coleta       95%          OK

Resumo da frota:
OK:        2 robo(s)
Atencao:   1 robo(s)
Critico:    1 robo(s)
```

Dica: declare os contadores zerados antes do loop e incremente-os dentro do bloco `if/elif/else` junto com a classificação do status.

Exercício 3

Crie o arquivo `diagnostico.sh`. O script simula um sistema de diagnóstico de um robô da frota W2G. Cada verificação deve ser encapsulada em uma função própria, e ao final o script consolida todos os resultados em um diagnóstico geral.

Os dados do robô devem ser definidos como variáveis no início do script:

```
robo="roboB"
bateria=15      # nível de bateria em %
sinal=45        # força do sinal de conexão em %
missao="emergencia"
```

O script deve implementar as quatro funções descritas abaixo e executá-las em sequência, exibindo o resultado de cada uma antes de apresentar o diagnóstico final.

Funções requeridas:

Função	Argumentos	O que faz
<code>verificar_bateria</code>	<code>bateria</code>	Classifica o nível de bateria e retorna um código de gravidade
<code>verificar_conexao</code>	<code>sinal</code>	Classifica a força do sinal e retorna um código de gravidade
<code>classificar_missao</code>	<code>missao</code>	Identifica o tipo de missão e retorna um código de gravidade
<code>diagnostico_geral</code>	<code>cod_bat</code> <code>cod_con</code> <code>cod_mis</code>	Soma os códigos e define o estado geral do robô

Critérios de classificação:

`verificar_bateria:`

Bateria	Texto retornado	Código
80% ou mais	OK	0
Entre 20% e 79%	ATENÇÃO	1
Abaixo de 20%	CRÍTICO	2

`verificar_conexao:`

Sinal	Texto retornado	Código
70% ou mais	Estável	0
Entre 30% e 69%	Instável	1
Abaixo de 30%	Sem sinal	2

`classificar_missao:`

Missão	Texto retornado	Código
<code>patrulha, mapeamento, coleta</code>	Operacional	0
<code>recarga, manutencao</code>	Em pausa	1
<code>emergencia</code>	Emergência	2
qualquer outro valor	Desconhecida	2

`diagnostico_geral:`

Soma dos códigos	Diagnóstico	Cor
0	Robô operando normalmente	Verde
1 ou 2	Robô requer atenção	Amarelo
3 ou mais	Robô em situação crítica	Vermelho

O resultado esperado para **roboB** com bateria **15**, sinal **45** e missão **emergencia** é mostrado na imagem abaixo:

```

diagnostico.sh

$ ./scripts/diagnostico.sh

=== Diagnostico: roboB ===

Bateria (15%):      CRITICO
Conexao (45%):      Instavel
Missao (emergencia): Emergencia

Robo em situacao critica

```

Exercício 4

Crie o arquivo **relatorio_frota.sh**. O script recebe os dados de cada robô diretamente pela linha de comando, no formato abaixo, e pode processar quantos robôs forem passados:

```
./relatorio_frota.sh <robo> <erros> <fatais> <criticos> [robo erros fatais criticos ...]
```

Exemplo com três robôs:

```
./relatorio_frota.sh roboA 2 0 0 roboB 8 2 2 roboZ 1 0 0
```

O script deve validar se os argumentos foram fornecidos corretamente. Cada robô requer exatamente quatro valores: nome, quantidade de erros, de fatais e de críticos. Se nenhum argumento for passado ou o total não for múltiplo de quatro, exibe uma mensagem de erro com o exemplo de uso e encerra.

O status de cada robô é definido com base na soma de **fatais** e **criticos**:

Condição	Status	Cor
Soma igual a 0	SAUDAVEL	Verde

Condição	Status	Cor
Soma entre 1 e 4	DEGRADADO	Amarelo
Soma igual ou acima de 5	CRITICO	Vermelho

Funções requeridas:

Função	Argumentos	O que faz
<code>validar_ambiente</code>	<code>\$@</code>	Verifica se os argumentos foram fornecidos e se o total é múltiplo de 4
<code>analisar_robo</code>	<code>robo erros fatais criticos</code>	Classifica o status, atualiza os contadores globais e chama <code>exibir_robo</code>
<code>exibir_robo</code>	<code>robo erros fatais criticos status cod</code>	Exibe o resumo colorido no terminal e salva no relatório
<code>exibir_resumo</code>	nenhum	Exibe e salva o resumo final com os contadores globais

Saída no terminal

A imagem abaixo mostra o resultado esperado no terminal ao executar o script com os três robôs de exemplo:

```
relatorio_frota.sh

$ ./relatorio_frota.sh roboA 2 0 0 roboB 8 2 2 roboZ 1 0 0

=== Relatorio Consolidado da Frota ===
Data: 2025-03-19 08:00:00

Robo: roboA   Status: SAUDAVEL
      error: 2   fatal: 0   critical: 0

Robo: roboB   Status: DEGRADADO
      error: 8   fatal: 2   critical: 2

Robo: roboZ   Status: SAUDAVEL
      error: 1   fatal: 0   critical: 0

=== Resumo Final ===
SAUDAVEL:  2 robo(s)
DEGRADADO: 1 robo(s)
CRITICO:   0 robo(s)
```

Salvamento do relatório

Ao final da execução, o script deve criar automaticamente uma pasta `relatorios/` no diretório onde está sendo executado e salvar o arquivo `relatorio_frota.txt` dentro dela. O conteúdo do arquivo deve seguir o formato abaixo, sem cores, pois é texto puro:

```
=== Relatorio Consolidado da Frota ===
Data: 2025-03-19 08:00:00

--- roboA ---
Status:    SAUDAVEL
error:     2
fatal:     0
critical:  0

--- roboB ---
Status:    DEGRADADO
error:     8
fatal:     2
critical:  2

--- roboZ ---
Status:    SAUDAVEL
```

```
error:      1
fatal:      0
critical:    0
```

```
=== Resumo Final ===
SAUDAVEL:   2 robo(s)
DEGRADADO:  1 robo(s)
CRITICO:    0 robo(s)
```